STRULE AUTOMATION
Industrial Controls · Bay Area & Northern California

PLANT CHECKLIST

Alarm Rationalization Worksheet: A Copy-and-Use Checklist for Killing Alarm Floods

Field notes from Jonathan Gilmour, independent controls engineer (PLC/HMI/SCADA), Bay Area and Northern California.

Most alarm systems I open up were never designed. They grew. Somebody added an alarm every time something went wrong and nobody ever took one away. Now the operator gets 400 alarms an hour, 30 of them in the first 10 seconds of any trip, and the one that actually mattered scrolled off the screen before anybody read it. That is an alarm flood, and it is how a good operator misses the alarm that would have saved the batch, the pump, or the shift.

This worksheet is the tool I use to fix that, one tag at a time. It pairs with the Strule blog post on alarm rationalization for food and beverage, but the method is cross-sector — it works the same on a digester skid, a boiler house, a packaging line, or a water plant.

Why floods happen, and what "good" looks like

The root cause is almost always the same: alarms were configured as *notifications* instead of as *calls to action*. If a point of information does not require the operator to do something, it is not an alarm — it is a log entry, a trend, or a status color on the screen.

The goal of rationalization is that **every alarm is actionable, prioritized, and gives the operator enough time to act before the consequence lands.** That is it. Three tests. If an alarm fails any one of them, you fix it or you delete it.

Two industry references frame this, and it is worth defining both because people throw the numbers around without them:

- **EEMUA-191** — the Engineering Equipment and Materials Users' Association Publication 191, "Alarm Systems: A Guide to Design, Management and Procurement." It is where the widely quoted performance targets come from: a target of roughly **one alarm every 10 minutes** per operator in steady state, and no more than about **10 alarms in the 10 minutes** after a major upset. Treat those as benchmarks to measure against, not a spec to hit on paper.

- **ISA-18.2** — the ANSI/ISA standard "Management of Alarm Systems for the Process Industries" (its international twin is IEC 62682). It defines the alarm *lifecycle* — identification, rationalization, design, implementation, operation, maintenance, monitoring, audit — and the idea that rationalization is a documented, repeatable step, not a one-time cleanup.

You do not need to buy either document to use this worksheet. You do need the mindset both share: an alarm is a promise that a human will do something, and the system owes that human enough time to do it.

Before you touch anything: two ground rules

Safety-related alarms and Safety Instrumented System (SIS) alarms are not on this worksheet for casual retuning. If an alarm is part of a protective function — a Safety Instrumented Function (SIF), a gas detection alarm, a high-high trip, an interlock permissive — you rationalize *whether the operator alarm is clear and actionable*, but you do not move the trip setpoint or defeat the function to make chatter go away. The legitimate move for a genuine nuisance is retuning the alarm (annunciation) setpoint or deadband **inside** the equipment and safety limit, one change at a time, with a written record. Changing a protective setpoint is a management-of-change action with its own review. Flag every one of these in the classification column and route it out of the everyday cleanup.

One change at a time, and log it. Every row you touch gets a date, the old value, the new value, who approved it, and why. If you cannot say why on one line, you are not ready to change it.

The per-alarm worksheet

Copy this table into your CMMS, a spreadsheet, or paper. One row per alarm tag. Work it top to bottom for each alarm — the order is deliberate; the actionability test comes first because it deletes work.

#	Field	What to write	How to decide it
1	**Tag / description**	The alarm tag (e.g. `FIT-2201 HI`) and a plain-language description an operator would recognize at 2am.	Pull it from the alarm database, not memory. Match the wording to what shows on the HMI.
2	**Is it actionable?** (Y/N)	Yes only if there is a specific operator action the alarm should trigger.	If No — there is nothing the operator can or should do — it is **not an alarm.** Convert it to an alert/log/status indication or delete it. Stop; do not fill in the rest of the row.

#	Field	What to write	How to decide it
3	**Consequence of inaction**	What actually happens if the operator ignores it — product loss, equipment damage, environmental release, safety event, downtime. Be concrete.	If you cannot name a real consequence, go back to row 2. "Someone might want to know" is not a consequence.
4	**Time available to respond**	The window from annunciation to the consequence landing. Seconds, minutes, or hours.	Estimate from the process: how fast does the level actually rise, the temperature actually run away? This number drives priority and setpoint.
5	**Correct operator action**	The single, clear thing the operator does. "Reduce feed and check strainer," not "investigate."	If the action is "call the supervisor" or "acknowledge," that is a sign it may not be a real alarm. One action, stated plainly.
6	**Priority**	Low / Medium / High (or Critical), matched to your HMI's priority scheme.	Set from consequence severity **and** time to respond — see the priority matrix below. Do not let everything become High; a system where 90% of alarms are High has no priority at all.
7	**Setpoint**	The value that trips the alarm, with units.	Set so the operator gets the time in row 4 — early enough to act, late enough that it is a real deviation, not normal operating noise. Check against the machine's spec and normal operating range; never invent an exact number that the process cannot support.
8	**Deadband / on-delay**	The reset gap and/or time delay that stops the alarm from chattering.	Deadband is the amount the signal must recover past the setpoint before the alarm resets, expressed as a percent of the instrument span. Typical starting points to check against your loop: roughly **1% of span** for a clean pressure or temperature signal, up to about **5% of span** for a noisy flow or level signal, plus an on-delay of a **few seconds** on noisy inputs. Tune to the actual signal noise — enough to stop chatter, not so much you eat the response time from row 4.

#	Field	What to write	How to decide it
9	**Classification**	Process / Equipment / Safety-related (SIS).	Process = deviation in the controlled variable. Equipment = a device fault (motor overload, VFD fault, comms). Safety-related = part of a protective function or gas/fire detection. **Safety-related rows do not get casually retuned** — route them to management-of-change.
10	**Change record**	Date, old value, new value, approver, reason.	Fill this in the moment you change anything. No entry, no change.

How to set priority (a simple matrix)

Priority is consequence severity crossed with how much time the operator has. Map it to however many levels your HMI actually supports — three working levels is plenty for most plants.

Time to respond ↓ / Severity →	Minor (log-worthy)	Serious (product/equipment)	Severe (safety/environmental)
Hours	Low	Low	Medium
Minutes	Low	Medium	High
Seconds	Medium	High	High / Critical

Two rules keep the scheme honest: - Aim for a distribution weighted toward Low — a healthy system runs about **80% Low, 15% Medium, 5% High**, the distribution EEMUA-191 points to. If yours is inverted, priorities are inflated. - If two alarms always come in together and mean the same operator action, they are one alarm. Suppress the follow-on or make it an alert.

Top offenders: the method that clears a flood fastest

You do not rationalize 2,000 alarms alphabetically. You go after the handful that generate most of the noise. In almost every system I audit, a small number of tags — often **fewer than 20** — produce well over half the annunciations. Kill or deband those and the operator can breathe.

Step by step:

1. **Pull the alarm log** — the alarm/event history from the DCS, SCADA, or HMI historian. Grab a representative window: at least a couple of weeks, and make sure it includes a normal run and at least one upset so you see both steady-state chatter and flood behavior.
2. **Rank every tag by frequency** — count of annunciations per tag over the window. Sort descending. This is your bad-actor list.

3. **Add a chatter check** — for the top tags, look at how many trips reset within a short window (say, under a minute). A tag that alarms and clears repeatedly is a deadband/delay problem, not a process problem.
4. **Work the top of the list first.** For each bad actor, run the per-alarm worksheet. Most of the top offenders fall into three buckets: - **Not actionable** → convert to alert/log or delete (row 2). - **Chattering** → add or widen deadband / add on-delay (row 8), inside limits, one change at a time. - **Bad setpoint** → the setpoint sits inside normal operating range, so it trips constantly → move it to a real deviation (row 7).
5. **Re-pull the log after each round** and re-rank. The list reshuffles as you fix the top. Two or three passes usually takes the flood out.
6. **Never silence a safety-related bad actor by defeating it.** If a gas detector or a high-high trip is chattering, that is a calibration, sensor, or process problem to fix at the source — not a deadband to widen past the safe limit.

Bad-actor tracking table:

Rank	Tag	Count (window)	Chatter? (Y/N)	Root cause bucket	Action taken	Re-check count
1						
2						
…						

Stale and standing-alarm check

Floods are the loud problem. Standing alarms are the quiet one, and they are just as dangerous — a screen full of alarms that have been active for hours or days trains the operator to ignore the whole list, including the new one that matters.

- **Standing alarm** — an alarm currently active (annunciated and not yet returned to normal).
- **Stale alarm** — a standing alarm that has stayed active a long time, commonly flagged at **active for 24 hours or more.**

Method:

1. From the current alarm summary, list every alarm that has been active longer than 24 hours (or your site's threshold).
2. For each, decide which it is:

Stale alarm cause	What it really is	Fix
Instrument failed / out of range	Bad transmitter, plugged line, dead sensor	Work order to repair; do not just suppress it and forget
Alarm is on a bypassed / out-of-service unit	Nuisance while the unit is down	Use a proper shelving/out-of-service function with an auto-return timer — not a permanent disable
Setpoint sits inside normal operating range	Never clears because it is always "in alarm"	Rationalize the setpoint (row 7) so it reflects a real deviation
Genuinely abnormal condition nobody is acting on	A real problem being ignored	Escalate — this is exactly the failure rationalization exists to prevent

3. **Shelving is not deleting.** If you take an alarm out of service, use the system's shelving/suppression feature with a time limit and a reason, so it comes back automatically and shows on a suppressed-alarm list. An alarm quietly turned off forever is how a plant loses protection it thinks it still has.
4. **Re-audit the stale list weekly** until it is short and stays short.

A note on getting help

None of this needs a big project. It needs someone to sit with the alarm log and the operators and go tag by tag with the discipline above. If your rationalization keeps stalling, it is usually because the alarms in question get handed back and forth between the machine vendor, the integrator, and the in-house team, and nobody owns the whole list. That hand-off is where an independent second look tends to earn its keep — one person tracking every change, inside the limits, on the record.

Work it one row at a time. Keep the change log. Re-pull the log and check your numbers against the EEMUA-191 benchmarks. When the operator can read the screen during an upset and knows exactly what each active alarm is asking them to do, you are done.